



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

JA. Swierczek
February 9th, 2025

Contact: JAS@TrueDataZ.com



Outline

- 3 [Executive Summary](#)
- 4 [Introduction](#)
- 5 [Methodology](#)
- 8 [Data Collection – SpaceX API](#)
- 17 [Insights Drawn from EDA](#)
- 34 [Launch Sites Proximity Analysis](#)
- 42 [Results of Predictive Analysis](#)
- 45 [Conclusions](#)
- 46 [Appendix](#)



Executive Summary

Methodology:

1. Data Collection: Used SpaceX REST API and Web Scraping from Wikipedia.
2. Data Wrangling: Cleaned data, handled missing values, and performed One-Hot Encoding.
3. EDA: Used SQL and Matplotlib/Seaborn for visualization.
4. Interactive Visual Analytics: Built maps with Folium and dashboards with Plotly Dash.
5. Predictive Analysis: Trained ML models like LogReg, SVM, KNN, Decision Trees ...
- *and research on Random Forest and LightGBM for further gains*

Results:

Drawing on EDA Results, Interactive Data Analytics, Predictive Analysis (Classification)

- Success rate has improved significantly over time
- KSC LC-39A is the most successful launch site. Whereas ES-L1, GEO, HEO, and SSO all have 100% success (with fewer launches): VLEO has better proven reliability
- Decision Tree models achieved the best Cross-Validation accuracy and with extra hypertuning maybe the best choices with such sparse, unbalanced data - however there are interpretability caveats
- All 4 ML models tested at 83.3% prediction accuracy...
- **We must consider more than Accuracy. We need emphasis also on Recall!**

Introduction

- **Project background and context**

- Space is becoming the new Wild West, with Private Enterprise now leading the field. The potential gains are literally Astronomical
- SpaceX is pack leader due to innovative solutions like the reusable rocket booster, which is a big fraction of total cost; Rocket Lab, Blue Origin, Relativity Space, Sierra Space, Firefly Aerospace, Astroscale – all are Billion dollar companies looking to stake their claims
- Few have what it takes to make deliveries to ISS, AND to outperform government Space Agencies on satellite launches... SpaceX may overextend by target fixating on Mars, leaving the door open a crack

- **Problems you want to resolve**

- The key factor guaranteeing the SpaceX competitive advantage is the enormous cost saving from each successfully returned stage one booster, and that is hard to beat
- To compete with SpaceX it is necessary to understand what goes in to a successful recovery, and to predict when that will or will not happen, and where...
- We have a lot of EDA and evaluation to do before we can predict the likely cost of SpaceX launches of the Falcon 9 and enable effective bidding in the market: Orbits, Payload Masses, FlightNumber, Sites...
- *The dataset combines many factors of relevance, from booster types to geographical choice of launch site, and payload factors to varieties of recovery, so we can engineer features – but the dataset is also not big... this is the **challenge** in training a model that both performs well in testing, and **still** generalizes on unseen data!*

Section 1

Methodology

Methodology

- Data collection methodology:
 - Using the SpaceX REST API
 - Using Web Scraping on Wikipedia Tables with BeautifulSoup
- Perform data wrangling
 - Data was processed using One Hot Encoding to generate binary choices; Null (NaN)/missing variables identified; Average values substituted; Irrelevant columns removed - the dataset was pruned to manageable dimensions of useful data
- Perform exploratory data analysis (EDA) using visualization and SQL Query
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models like LogReg, SVM and KNN, Decision Trees... (also trying Random Forest and LightGBM)
 - Building, tuning, and evaluating the classification models by splitting data into training and unexposed test sets
 - Determining optimal hyperparameters and testing models for Accuracy, Recall and generalizability

Data Collection

Data Source 1: SpaceX REST API

- Endpoint: <https://api.spacexdata.com/v4>
- Collected Rocket, Launch, Payload, and Core data

Data Source 2: Web Scraping

- Source: Wikipedia 'List of Falcon 9 and Falcon Heavy launches'
- Method: Scraped HTML tables using BeautifulSoup4

Request SpaceX Data:

```
spacex_url="https://api.spacexdata.com/v4/launches/past"
response = requests.get(spacex_url)
```

Use json_normalize method converting .json() content to Pandas dataframe:

```
data=pd.json_normalize(response.json())
```

Query API

Custom Functions: def...
getBoosterVersion(data),
getPayloadData(data),
getLaunchSite(data),
getCoreData(data)

Filter to include only Falcon 9:

```
data_falcon9 = data_launch[data_launch['BoosterVersion'] != 'Falcon 1']
```

Create Pandas DF from Dictionary:

```
data_launch = pd.DataFrame.from_dict(launch_dict)
```

Combine the columns into a dictionary:
launch_dict, then ...

Data Collection – SpaceX API



- Data collection with SpaceX REST calls - see key phrases and flowchart above:
- Process:
 1. Request data from API endpoints
 2. Convert JSON response to Pandas DataFrame
 3. Filter for Falcon 9 launches only
- **Key Features Extracted:**
 - - FlightNumber, Date, BoosterVersion, PayloadMass
 - - Orbit, LaunchSite, Outcome, Flights, GridFins
 - - Reused, Legs, LandingPad, Block, Serial

**Data Wrangle
missing
rows:**

```
data_falcon9.isnull  
().sum()
```

Create CSV

• **CSV**

```
data_falcon9.to_  
csv('dataset_p  
art_1.csv',  
index=False)
```

• **Replace NaN
with Mean:**

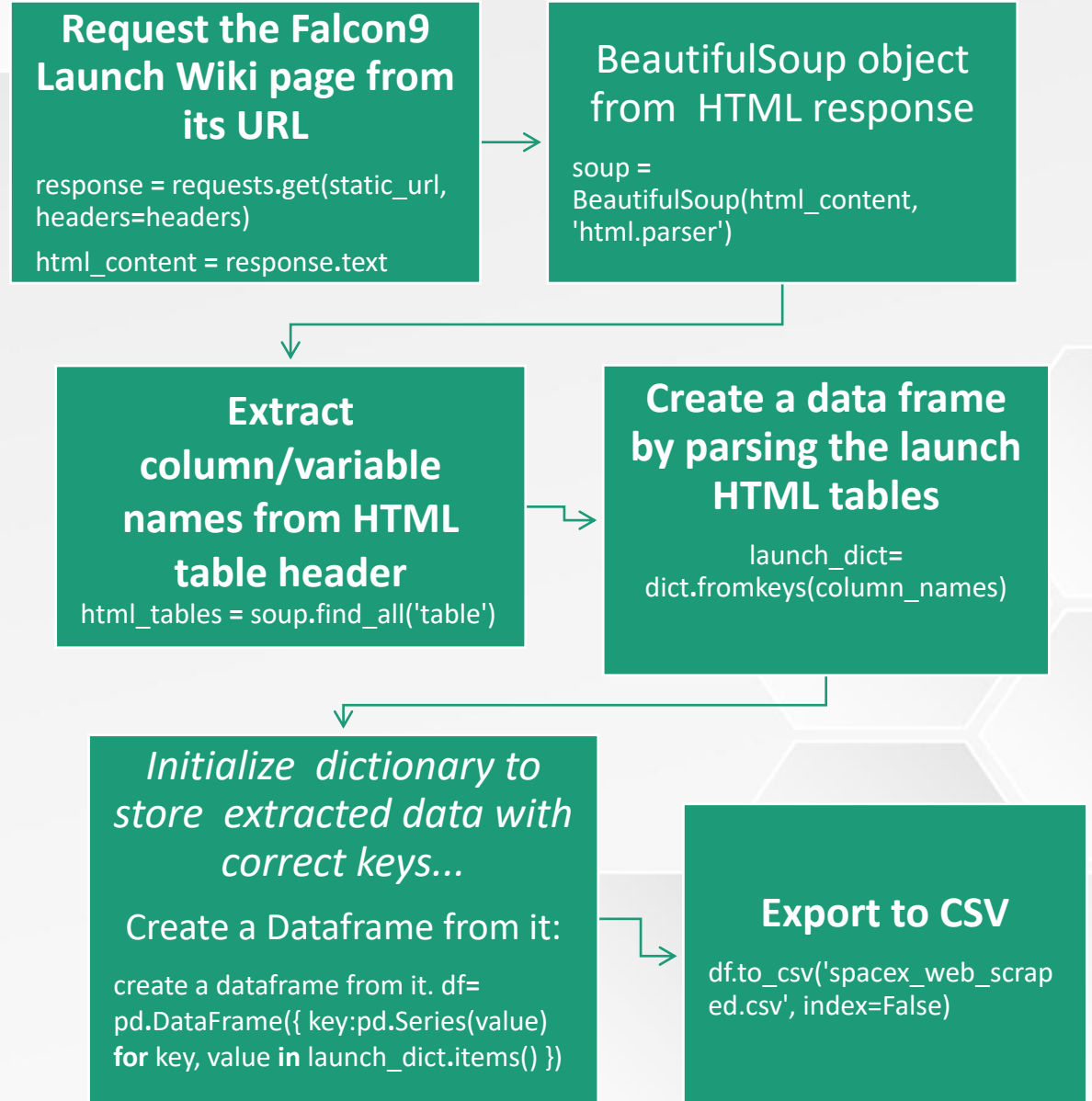
```
# Calculate the mean value  
of PayloadMass column  
mean_payload_mass =  
data_falcon9['PayloadMass'  
,mean()]
```

```
# Replace the np.nan  
values with its mean  
value  
data_falcon9['PayloadMa  
ss'].replace(np.nan,  
mean_payload_mass,  
inplace=True)
```


Data Collection - Scraping

- Web scraping process:
 1. Fetch Wikipedia page content using `requests`
 2. Parse HTML using `BeautifulSoup4`.
 3. Iterate through tables to extract launch records
 4. Clean and structure data into a DataFrame
- Helper functions: **def**
`date_time(table_cells)`
`booster_version(table_cells)`
`landing_status(table_cells)`
`get_mass(table_cells)`

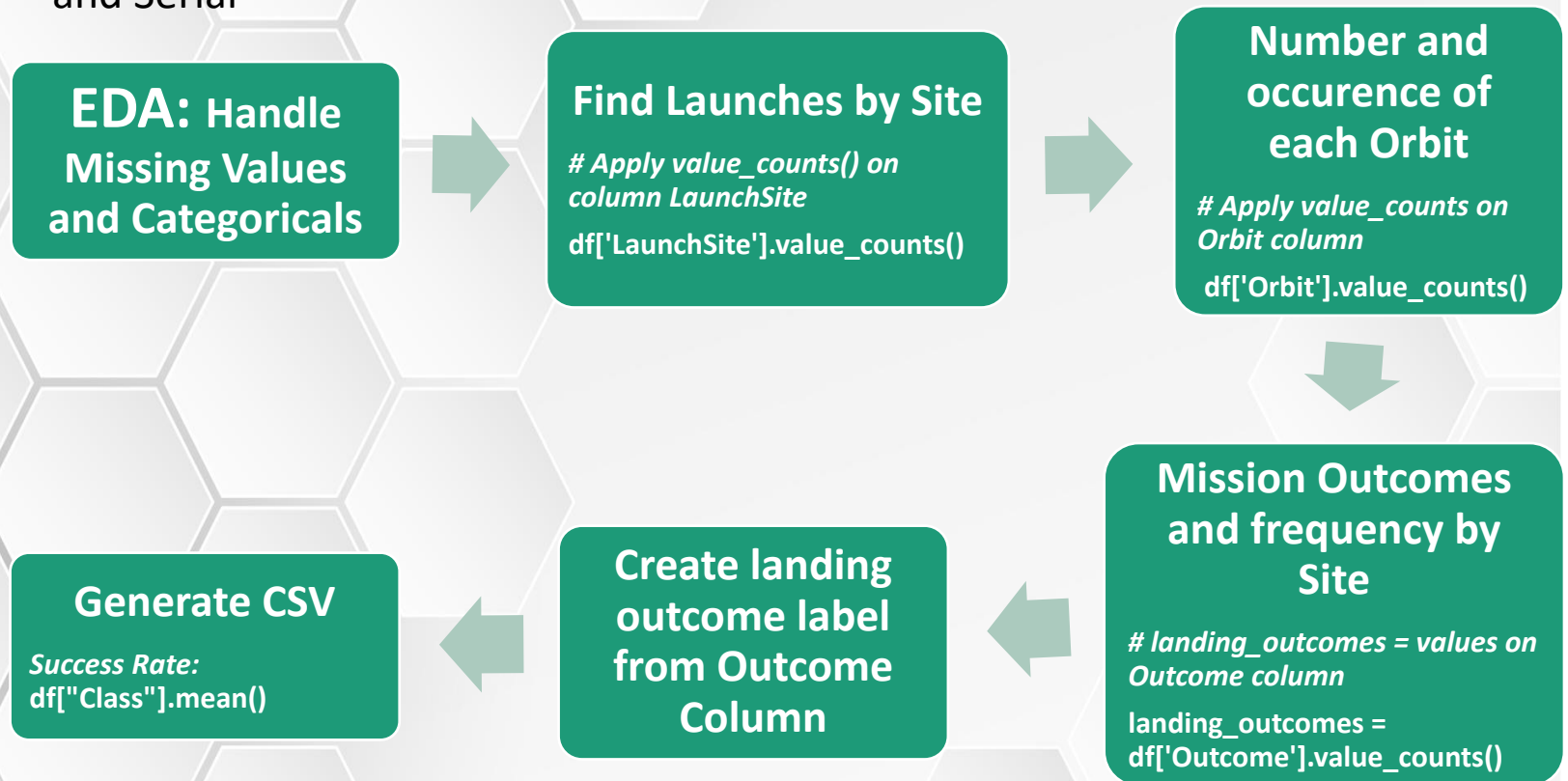
[Web scraping](#)



Data Wrangling

- Data wrangling process in key phrases and flowcharts:

1. Filtering: Retained only Falcon 9 launches
2. Missing Values: Imputed missing PayloadMass with the mean
3. Class Label: Created binary 'Class' column (1 = Success, 0 = Failure)
4. Categorical Data: Applied One-Hot Encoding to Orbit, LaunchSite, LandingPad, and Serial



EDA with Data Visualization

- Visualization:

Scatter plots (Payload vs. Site) highlight the relationships between variables much needed for machine learning methods

Bar charts (Success vs. Orbit) compare discrete categories or groups very clearly

Line charts (Success Rate vs. Year) show trends in data (time series analysis)

Key Insights: we can quickly SEE...

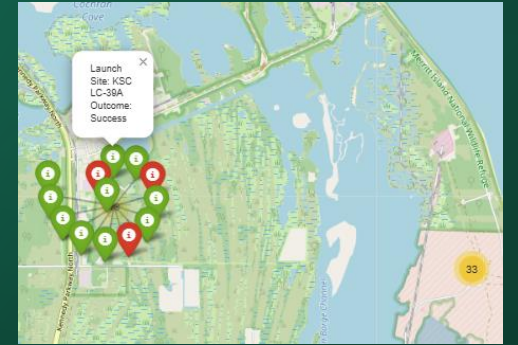
- *KSC LC-39A has the highest launch volume and success rate*
- *Success rate also correlates with Payload Mass, and Flight Number*
- *Orbits ES-L1, GEO, HEO, and SSO: 100% success rates, VLEO close behind*

EDA with SQL

EDA with SQL



Build an Interactive Map with Folium



01

Mouseover tooltips for details

Markers with circle, popup label & text for Launch Sites & Coordinates

Marker Clusters in green and red

Coloured distance calculator lines to nearest features, infrastructure and city

02

Mouseover Tooltips: quick detail access

Markers: for graphical clarity. More detail appears on zooming for ease of interpretation

Color-coded: Green (Success) / Red (Failure) at Launch Sites. Facilitates analysis directly from graphs

Coloured distance lines: for Launch Sites to nearest coastlines, etc. (for safety) further enabling access planning

03

Launch Site Locations - Graphical Mapping

Build a Dashboard with Plotly Dash

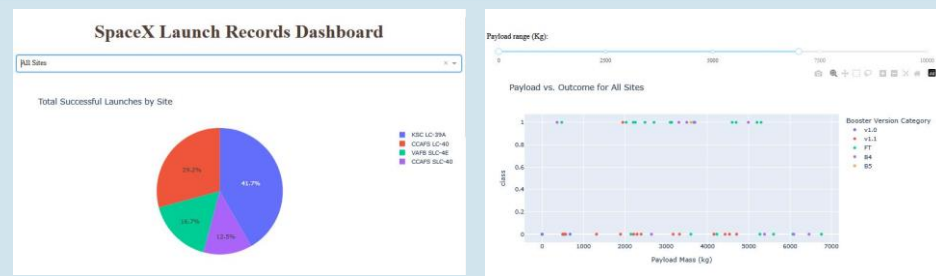
Summarize

Launch Sites: Dropdown

Pie Chart: Launch Success

Scatter: Pay Load by Success Rate

Slider: Payload Ranges



Explain

1. For ease of navigation starts with “All Sites” - Quick access to individual site results
2. Visually, the pie chart best delivers static success and failure data for all, and for each of the the four sites
3. The payload data covers a wide range of values and is very different between sites; It is valuable to be able to see the impact of these ranges controlled using a slider
4. Visually, the correlation is well explored using the slider and scatter, varying interactively

Add

- 1: Add a dropdown list to enable Launch Site selection - default set to ALL sites; For specific launch site selections, show "Success vs. Failed" counts
- 2: Add a pie chart showing total successful launches count for all sites
- 3: Add a slider to select payload range
- 4: Add a scatter plot to show correlation between payload (slider for ranges) and launch success

Predictive Analysis (Classification)

Load prepared dataframe - Separate out the independent variable standardized as X, (make Y array) - Fit and Transform X using StandardScaler, then ***"X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=nn)"*** - use same nn seed for replicability

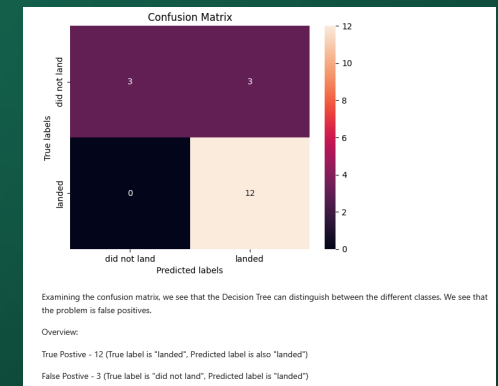


For each model create a GridSearchCV object - fit it to find best parameters from the dictionary. Display these using data attribute ***"best_params_"*** and accuracy on validation data using data attribute ***"best_score_"***



From gridsearch ***best_model = «modelname_cv».best_estimator_*** and ***Y_pred = best_model.predict(X_test)*** - display test accuracy & best hyperparameters using «score» - then ***plot_confusion_matrix(Y_test,yhat)*** where ***yhat=«modelname_cv».predict(X_test)*** Finally assessments - we tend to False Positives with these four models - worth trying Random Forest, LightGBM

Results



1

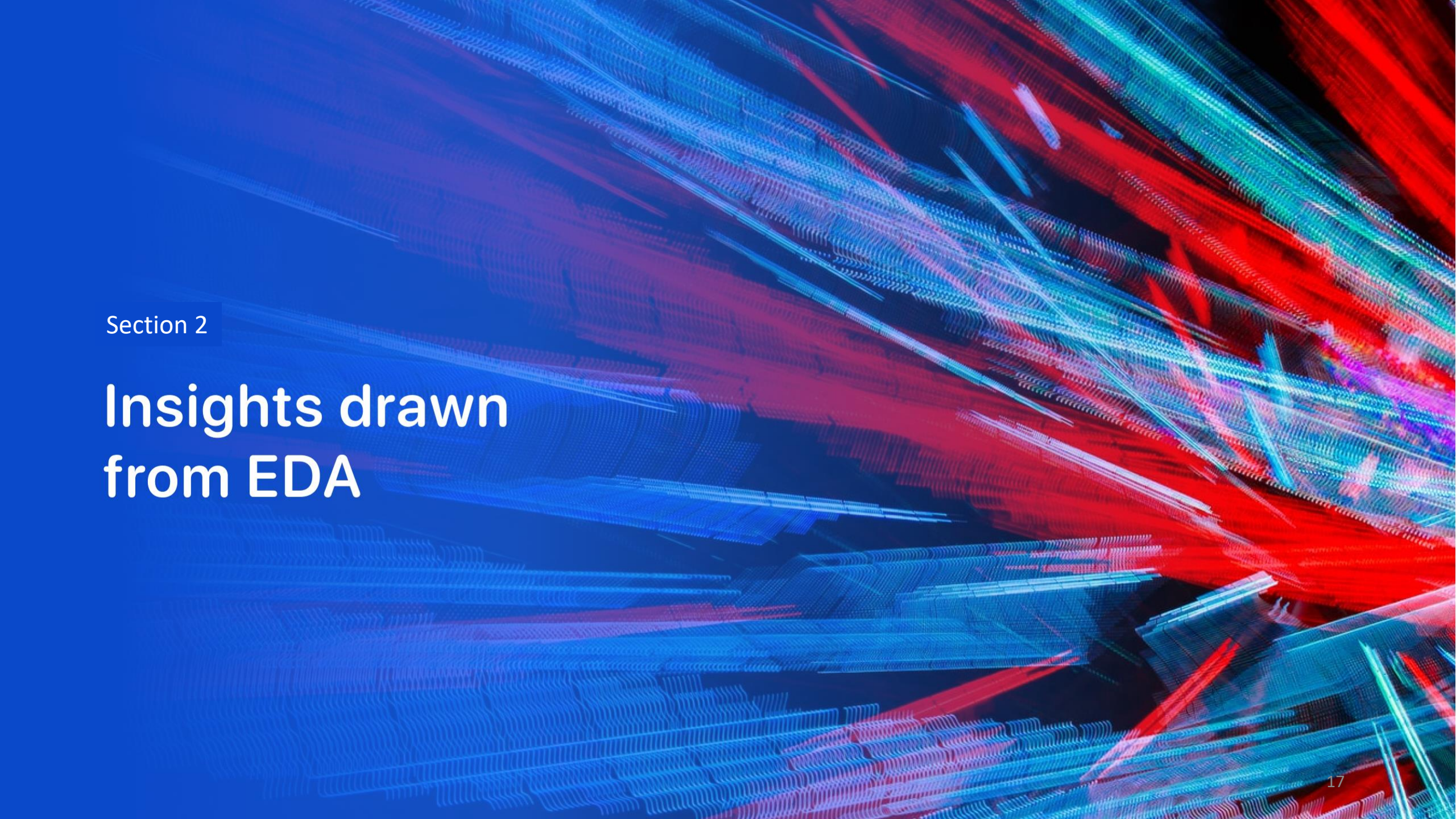
Exploratory data
analysis results

2

Interactive analytics
demo in screenshots

3

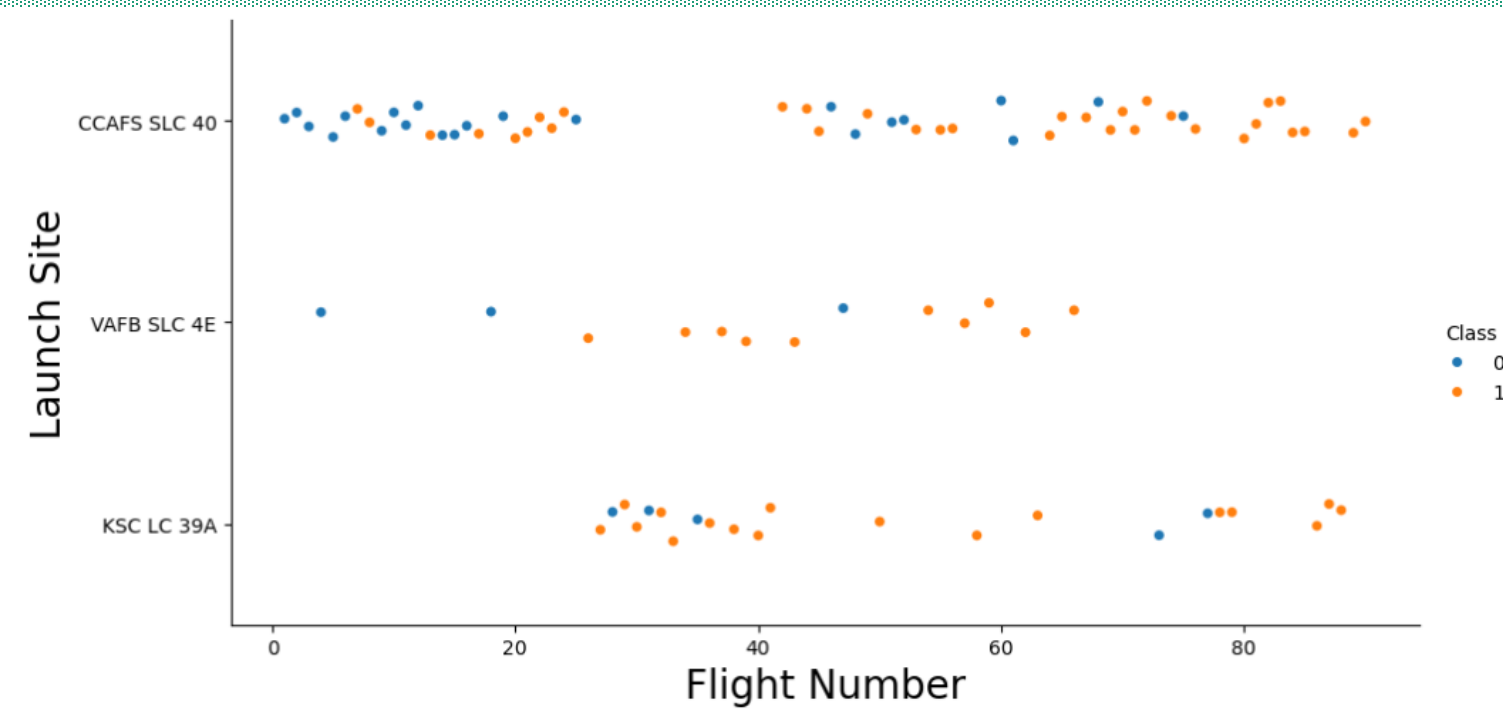
Predictive analysis
results



Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

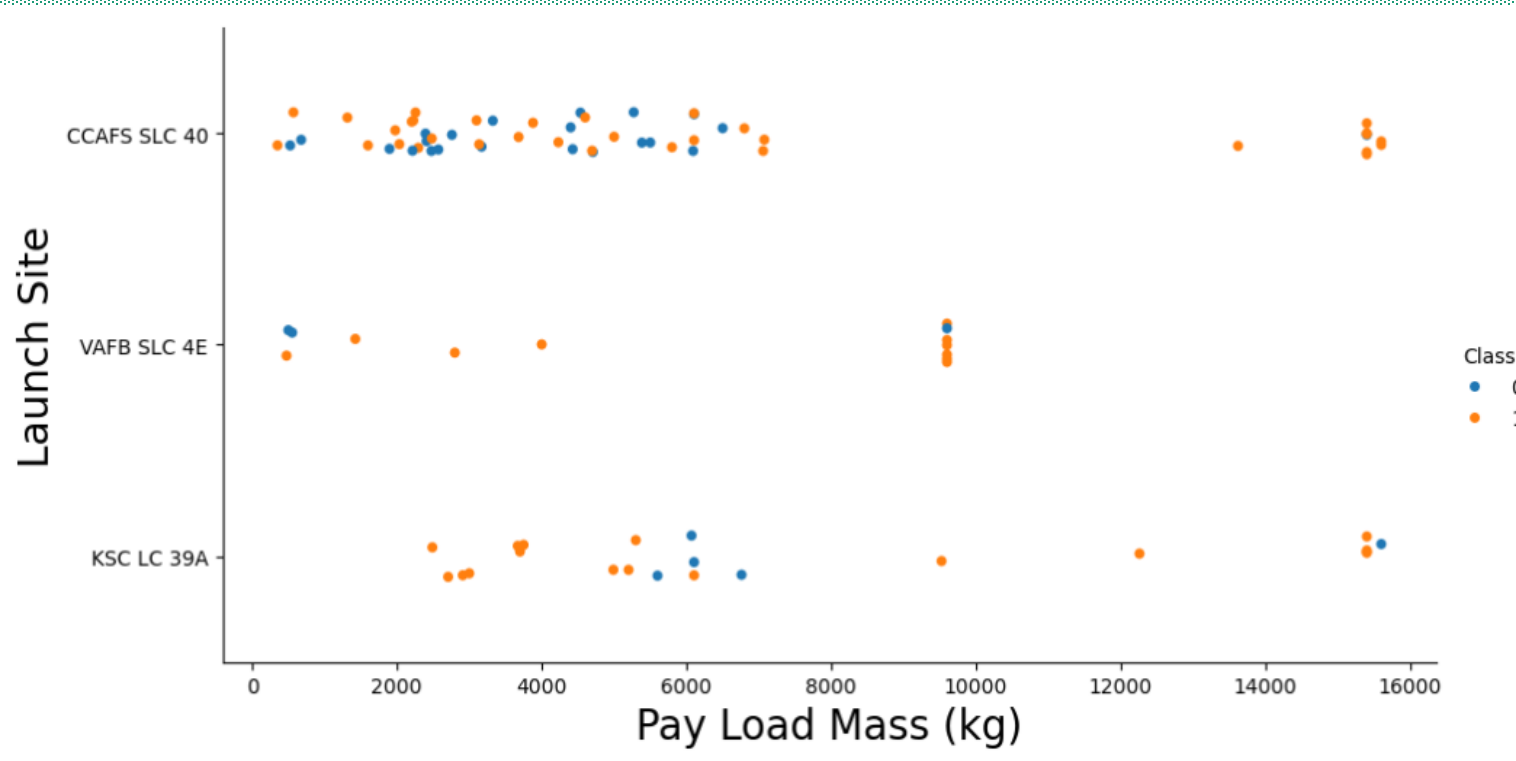


```
sns.catplot(y="LaunchSite",  
x="FlightNumber", hue="Class", data=df,  
aspect=2)  
plt.xlabel("Flight Number", fontsize=20)  
plt.ylabel("Launch Site", fontsize=20)  
plt.show()
```

We plot a scatter point chart with x-axis as FlightNumber and y-axis as LaunchSite, and with hue as Class value

We want to observe if there is any relationship between Launch Sites and their Flight Numbers

Payload vs. Launch Site

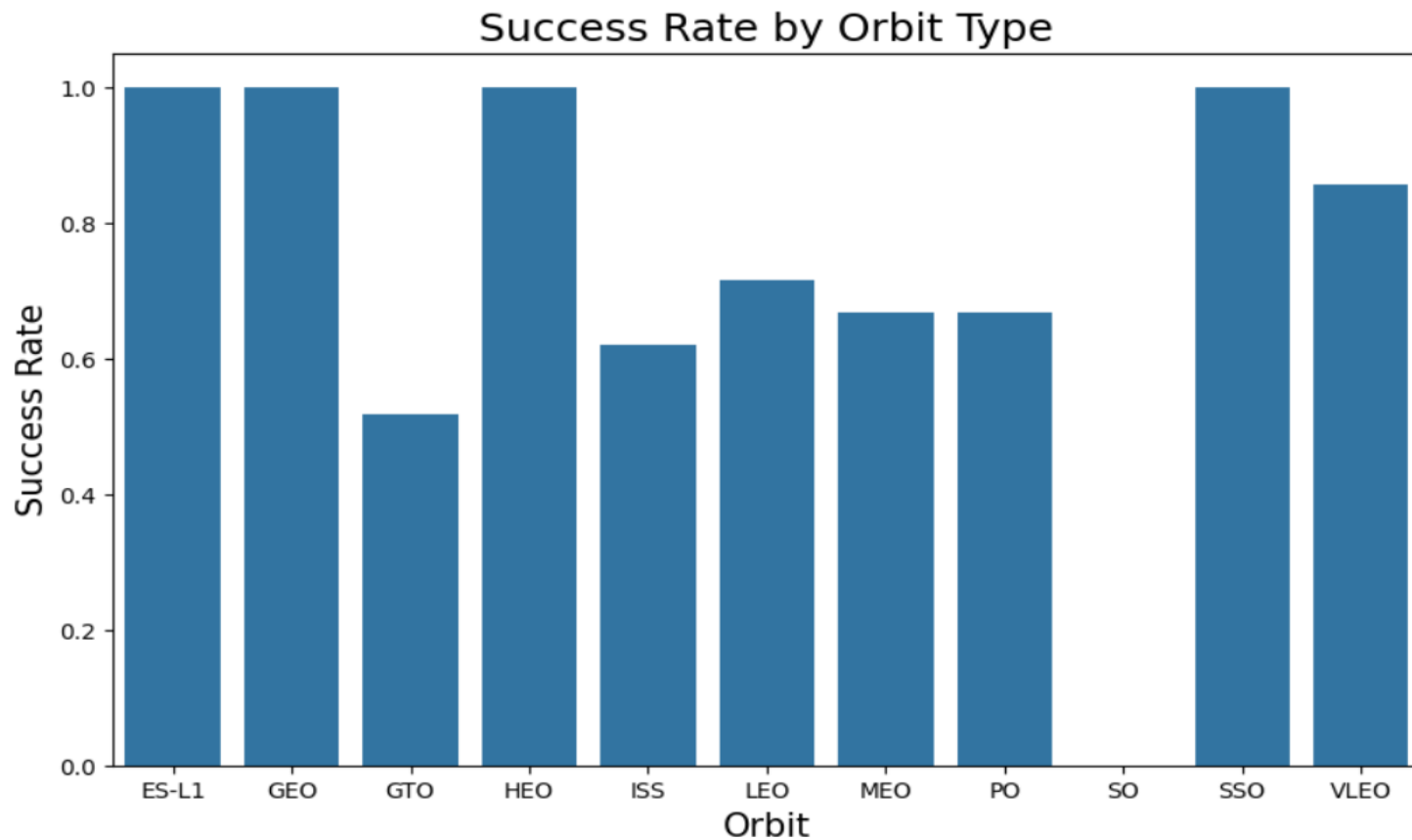


```
sns.catplot(y="LaunchSite",  
x="PayloadMass", hue="Class", data=df,  
aspect=2)  
plt.xlabel("Pay Load Mass (kg)", fontsize=20)  
plt.ylabel("Launch Site", fontsize=20)  
plt.show()
```

We also want to observe if there is any relationship between Launch Sites and their Pay Load Mass (so we plot a scatter point chart)

We see that for the VAFB-SLC launchsite there are no rockets launched for heavy Pay Load Mass (greater than 10000)

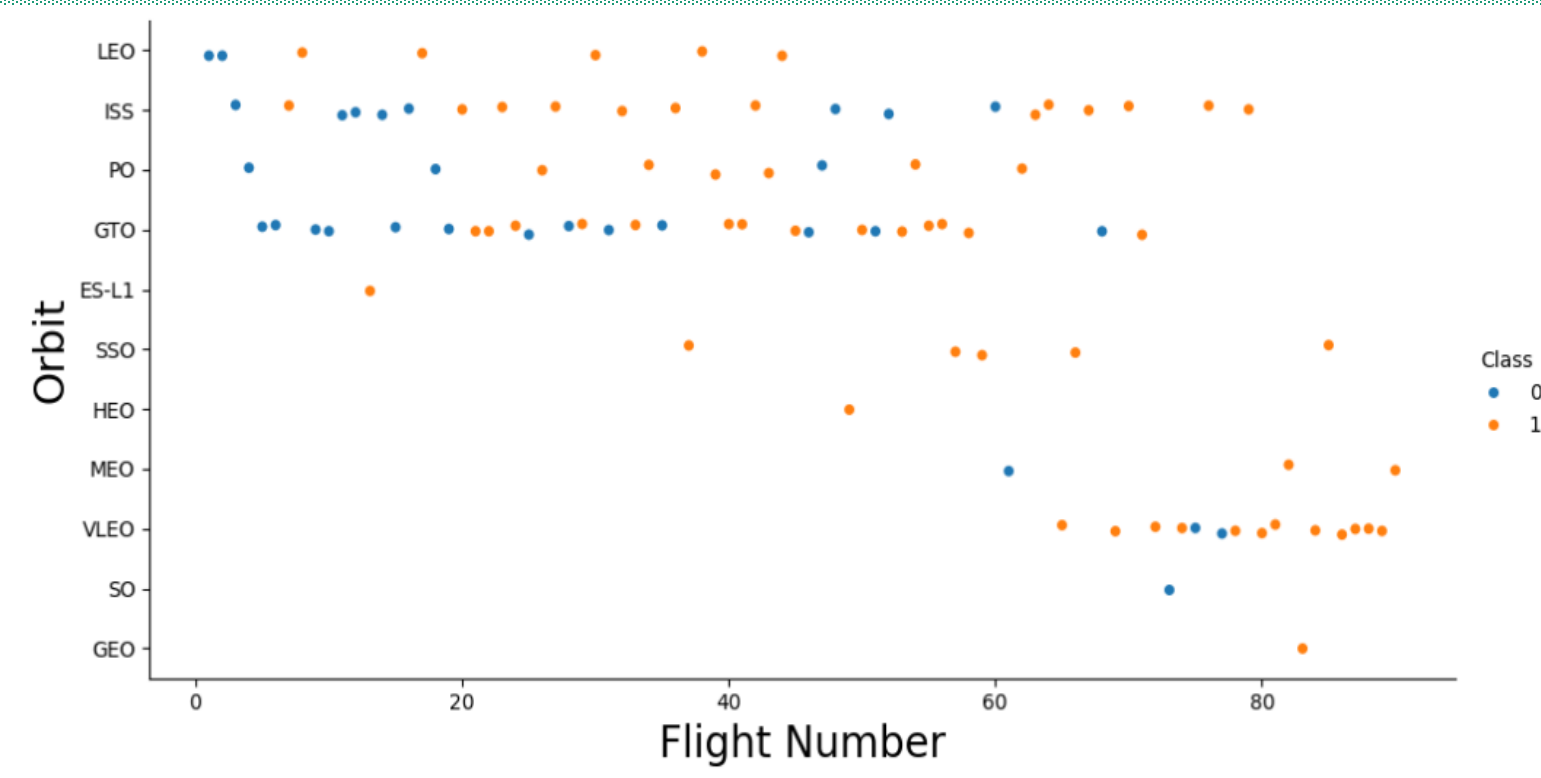
Success Rate vs. Orbit Type



```
# orbit_success_rate =  
df.groupby('Orbit')['Class'].mean().reset_index()  
plt.figure(figsize=(10, 6))  
sns.barplot(x="Orbit", y="Class",  
data=orbit_success_rate)  
plt.xlabel("Orbit", fontsize=15)  
plt.ylabel("Success Rate", fontsize=15)  
plt.title("Success Rate by Orbit Type", fontsize=18)  
plt.show()
```

Next, we want to visually check if there are any relationships between success rate and orbit type. Let's create a bar chart for the success rate of each orbit: ES-L1, GEO, HEO, SSO perform best, then VLEO

Flight Number vs. Orbit Type

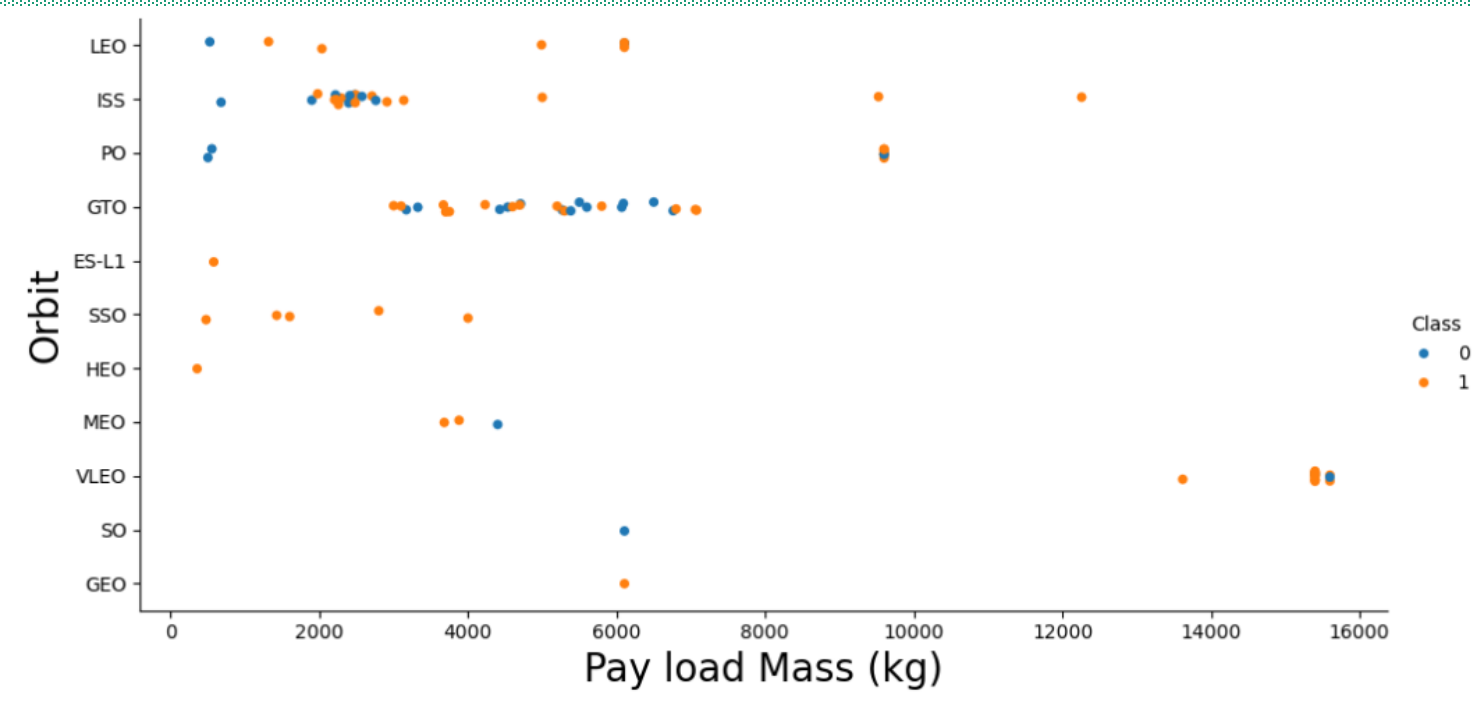


```
sns.catplot(y="Orbit", x="FlightNumber",  
hue="Class", data=df, aspect=2)  
plt.xlabel("Flight Number", fontsize=20)  
plt.ylabel("Orbit", fontsize=20)  
plt.show()
```

For each orbit, we want to see if there is any relationship between FlightNumber and Orbit type (so we plot a scatter point chart)

We observe that in the LEO orbit data, success is related to the number of flights. Whereas, in the GTO orbit, there appears to be no relationship between flight number and success

Payload vs. Orbit Type

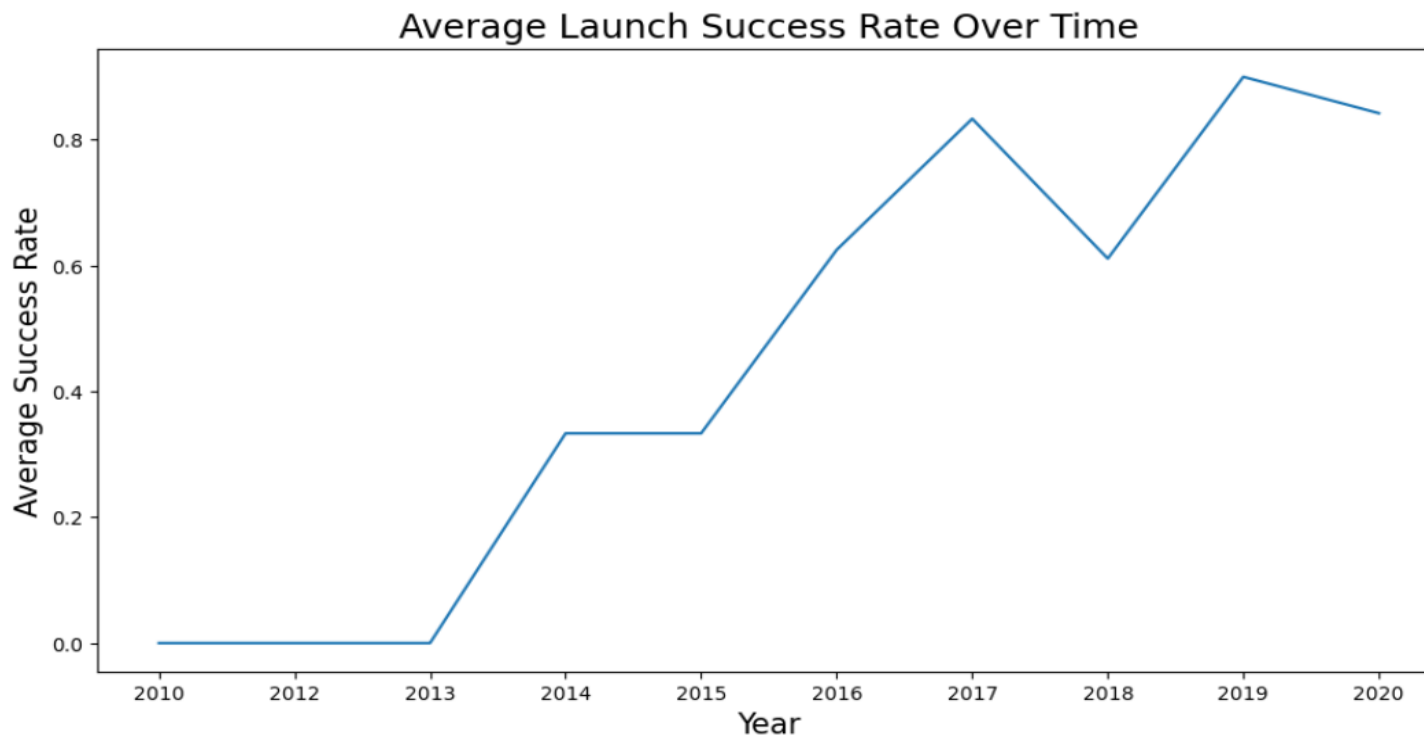


```
sns.catplot(y="Orbit", x="PayloadMass",  
hue="Class", data=df, aspect=2)  
plt.xlabel("Pay load Mass (kg)", fontsize=20)  
plt.ylabel("Orbit", fontsize=20)  
plt.show()
```

Similarly, we can plot Payload Mass vs. Orbit scatter point charts to reveal the relationship between Payload Mass and Orbit type

With **heavy payloads** successful landings are more for LEO, ISS, VLEO and PO(lar). For GTO it's difficult to distinguish, as both outcomes are present

Launch Success Yearly Trend



The success rate increases from 2013 - 2020

```
df['Date'] = Extract_year(df)
success_rate_by_year =
df.groupby('Date')['Class'].mean().reset_index()

plt.figure(figsize=(12, 6))
sns.lineplot(x="Date", y="Class",
data=success_rate_by_year)
plt.xlabel("Year", fontsize=15)
plt.ylabel("Average Success Rate", fontsize=15)
plt.title("Average Launch Success Rate Over
Time", fontsize=18)
plt.show()
```

We plot a line chart with x-axis as the extracted year and y-axis as the success rate

All Launch Site Names

Display the names of the unique launch sites in the space mission

```
%%sql
SELECT DISTINCT "Launch_Site" FROM SPACEXTABLE;
```

```
* sqlite:///my_data1.db
```

Done.

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

SELECT DISTINCT "Launch_Site" from SPACEXTABLE;

Queries the SPACEXTABLE for DISTINCT entries of the type "Launch_Site" and returns the list:

- CCAFS LC-40 *Cape Canaveral Air Force Station*
- VAFB SLC-4E *Vandenberg Air Force Base SLC-4E*
- KSC LC-39A *Kennedy Space Center "Launch Complex" 39A*
- CCAFS SLC-40 *Cape Canaveral Space Launch Complex 40 "Slick Forty"*

Launch Site Names Begin with 'CCA'

Task 2

Display 5 records where launch sites begin with the string 'CCA'

```
%%sql
SELECT "Launch_Site"
FROM SPACEXTABLE
WHERE "Launch_Site" LIKE 'CCA%'
LIMIT 5;
```

```
* sqlite:///my_data1.db
Done.
```

Launch_Site

CCAFS LC-40

CCAFS LC-40

CCAFS LC-40

CCAFS LC-40

CCAFS LC-40

```
SELECT "Launch_Site" from SPACEXTABLE
WHERE "Launch_Site" LIKE 'CCA%'
LIMIT 5;
```

Outputs the top 5 results from the query to SPACEXTABLE asking WHERE it should SELECT entries given the condition LIKE, implying similarity as specified in 'CCA%' – so all things beginning CCA

There are five returns all CCAFS LC-40 (Cape Canaveral Air Force Station – Launch Complex 40)

Total Payload Mass

Display the total payload mass carried by boosters launched by NASA (CRS)

```
%%sql
SELECT SUM("Payload_mass__kg_") AS Total_Payload_Mass
FROM SPACEXTABLE
WHERE "Customer" = 'NASA (CRS)';
```

```
* sqlite:///my_data1.db
```

Done.

Total_Payload_Mass

45596

```
SELECT SUM("Payload_mas__kg_" AS
Total_Payload_Mass from SPACEXTABLE
WHERE "Customer" = 'NASA (CRS)';
```

Queries SPACEXTABLE for all entries with NASA (CRS) as Customer, and forms a SUM of "Payload_mass__kg_" storing this as Total_Payload_Mass

45596 kg is the resulting total

Average Payload Mass by F9 v1.1

Display average payload mass carried by booster version F9 v1.1

```
%%sql
SELECT AVG("Payload_mass__kg_") AS Average_Payload_Mass
FROM SPACEXTABLE
WHERE "Booster_Version" = 'F9 v1.1';
```

```
* sqlite:///my_data1.db
Done.
```

Average_Payload_Mass

2928.4

```
SELECT AVG("Payload_mass__kg_") AS
Average_Payload_Mass
from SPACEXTABLE
WHERE "Booster_Version" = 'F9 v1.1';
```

Queries SPACEXTABLE only for the given Booster_Version, and then forms an AVG of the Payload_mass__kg_ data saving the result to Average_Payload_Mass

2928.4 kg is the average of Payload Mass for the F9 v1.1 Booster

First Successful Ground Landing Date

List the date when the first succesful landing outcome in ground pad was acheived.

```
%%sql
SELECT MIN(Date) AS First_Successful_Ground_Pad_Landing
FROM SPACEXTABLE
WHERE "Landing_Outcome" = 'Success (ground pad)';
```

```
* sqlite:///my_data1.db
```

Done.

First_Successful_Ground_Pad_Landing

2015-12-22

```
SELECT MIN(date) AS
First_Successful_Ground_Pad_
Landing
From SPACEXTABLE
WHERE "Landing_Outcome" =
'Success (ground pad)';
```

Queries SPACEXTABLE for every 'Success (ground pad)' of "Landing_Outcome" then saves the MIN of the dates as First_Successful_Ground_Pad_Landing: 2015-12-22

Successful Drone Ship Landing with Payload between 4000 and 6000

F9 FT B1022

SELECT "Booster_Version" FROM SPACEXTABLE

F9 FT B1026

WHERE "Landing_Outcome" = 'Success (droneship)'

F9 FT B1021.2

AND "PAYLOAD_MASS__KG_" > 4000

F9 FT B1031.2

AND "PAYLOAD_MASS__KG_" < 6000;

SELECT "Booster_Version" FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (droneship)' AND "PAYLOAD_MASS__KG_" > 4000 AND "PAYLOAD_MASS__KG_" < 6000;

Queries SPACEXTABLE for all Booster_Version carrying 6000 < PAYLOAD_MASS__kg < 4000kg with Landing_Outcome in Success for droneship landings

```
List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

%%sql
SELECT "Booster_Version"
FROM SPACEXTABLE
WHERE "Landing_Outcome" = 'Success (drone ship)'
AND "PAYLOAD_MASS__KG_" > 4000
AND "PAYLOAD_MASS__KG_" < 6000;

* sqlite:///my_data1.db
Done.

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2
```

Total Number of Successful and Failure Mission Outcomes

List the total number of successful and failure mission outcomes

```
%%sql
SELECT "Mission_Outcome", COUNT("Mission_Outcome") AS Total
FROM SPACEXTABLE
GROUP BY "Mission_Outcome";
```

* sqlite:///my_data1.db

Done.

Mission_Outcome	Total
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

```
SELECT "Mission_Outcome",
COUNT("Mission_Outcome") AS Total FROM
SPACEXTABLE
GROUP BY "Mission_Outcome";
```

Queries SPACEXTABLE to count the Mission_Outcome as Total of these, then provide the GROUP as a result of all Mission_Outcome totals

Only 1 failure (in flight) and 1 unclear payload status recorded

Boosters Carried Maximum Payload

List all the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

```
%%sql
SELECT "Booster_Version"
FROM SPACEXTABLE
WHERE "PAYLOAD_MASS__KG_" = (
    SELECT MAX("PAYLOAD_MASS__KG_")
    FROM SPACEXTABLE
);
```

```
* sqlite:///my_data1.db
Done.
```

Booster_Version

F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

```
SELECT "Booster_Version" FROM
SPACEXTABLE WHERE
"PAYLOAD_MASS__KG_" = (SELECT
MAX("PAYLOAD_MASS__KG_")
FROM SPACEXTABLE);
```

Queries SPACEXTABLE for instances of Booster_Version with their given PAYLOAD_MASS__kg_ then Queries AGAIN in SPACEXTABLE to SELECT only those with the MAX value for these Payload entries

Results in a list of 12: all varieties of F9 B5 B1

2015 Launch Failure Records

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

Note: SQLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

```
%%sql
SELECT
    substr(Date, 0, 5) AS Year,
    substr(Date, 6, 2) AS Month,
    "Booster_Version",
    "Launch_Site",
    "Landing_Outcome"
FROM SPACEXTABLE
WHERE substr(Date, 0, 5) = '2015'
    AND "Landing_Outcome" = 'Failure (drone ship)';
```

* sqlite:///my_data1.db

Done.

Year	Month	Booster_Version	Launch_Site	Landing_Outcome
2015	01	F9 v1.1 B1012	CCAFS LC-40	Failure (drone ship)
2015	04	F9 v1.1 B1015	CCAFS LC-40	Failure (drone ship)

```
SELECT substr(Date, 0, 5) AS
Year, substr(Date, 6, 2) AS
Month, "Booster_Version", "Launch_Site",
"Landing_Outcome" FROM SPACEXTABLE
WHERE substr(Date, 0, 5) = '2015' AND
"Landing_Outcome" = 'Failure (drone ship)';
```

Queries SPACEXTABLE using subst for Months of Year 2015 with Landing_Outcome = Failures of drone ship, and Booster_Version, Launch_Site, Landing_Outcome as columns. So all 2015 failures...

January and April 2015 from CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

```
SELECT "Landing_Outcome",  
COUNT("Landing_Outcome") AS Total_Count  
FROM SPACEXTABLE  
WHERE Date BETWEEN '2010-06-04'  
AND '2017-03-20'  
GROUP BY "Landing_Outcome"  
ORDER BY Total_Count DESC;
```

Queries SPACESTABLE in a specified date range counting all Landing_Outcome to save as Total_Count, then Grouping these to display in Descending order

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
%%sql  
SELECT "Landing_Outcome", COUNT("Landing_Outcome") AS Total_Count  
FROM SPACEXTABLE  
WHERE Date BETWEEN '2010-06-04' AND '2017-03-20'  
GROUP BY "Landing_Outcome"  
ORDER BY Total_Count DESC;
```

```
* sqlite:///my_data1.db  
Done.
```

Landing_Outcome	Total_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

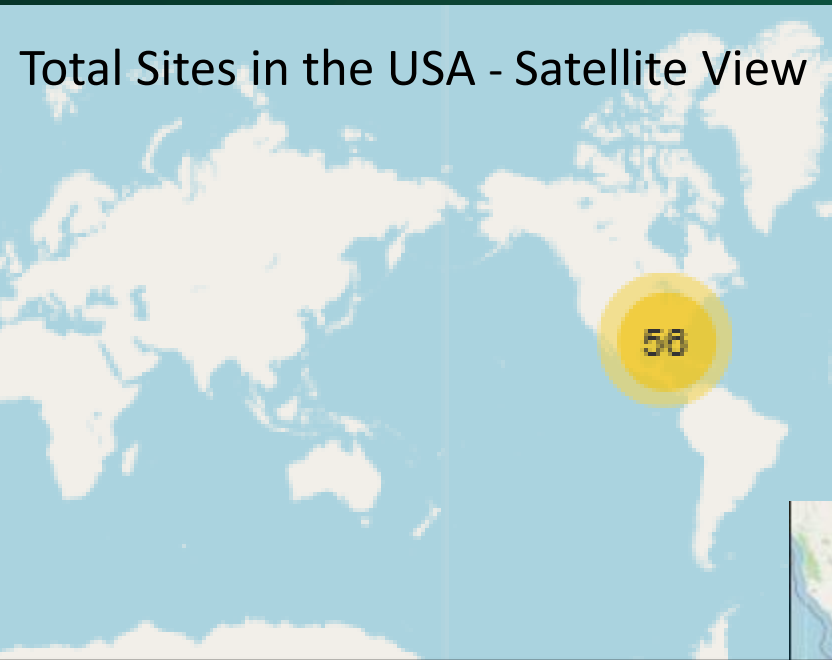
A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The background is a deep blue gradient.

Section 3

Launch Sites Proximities Analysis

Folium Map: Global Map Views

Total Sites in the USA - Satellite View

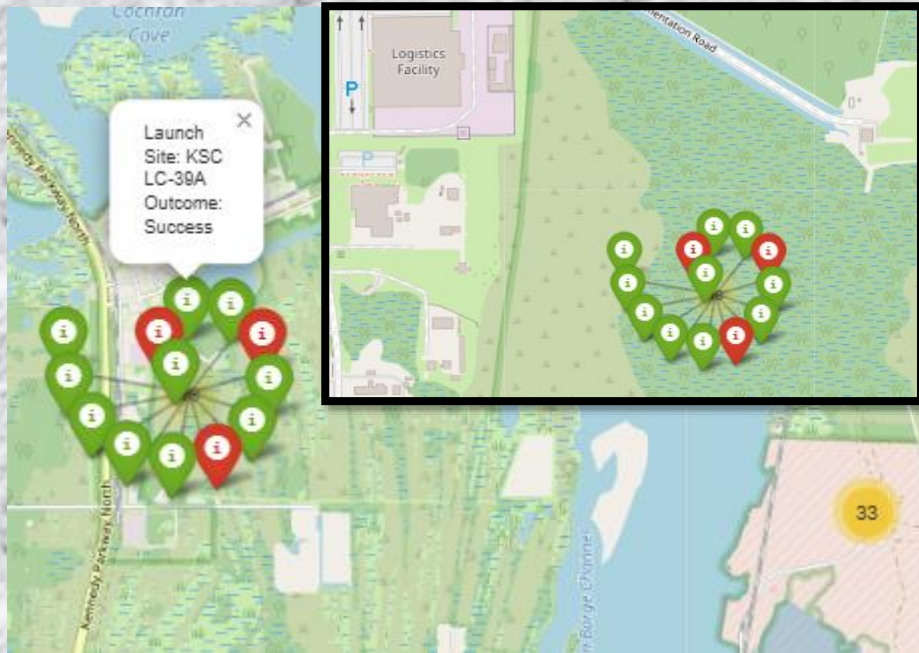


Zooming anticklockwise. From World View, through USA View, to the exploded view out of Vandenberg AFB, finally, to Florida and the next level of Zoom, showing Cape Canaveral and KSC totals. From the Full World to individual Launch Sites in only three clicks of Zoom

Coast to Coast



Folium Map: Launch Outcomes View



These views do not include
Vandenberg AFB

To the left, the 13 Outcomes are shown for Kennedy SC (zoomed in view in picture)

To the right, the 33 Outcomes split over the two Cape Canaveral sites (26 & 7)

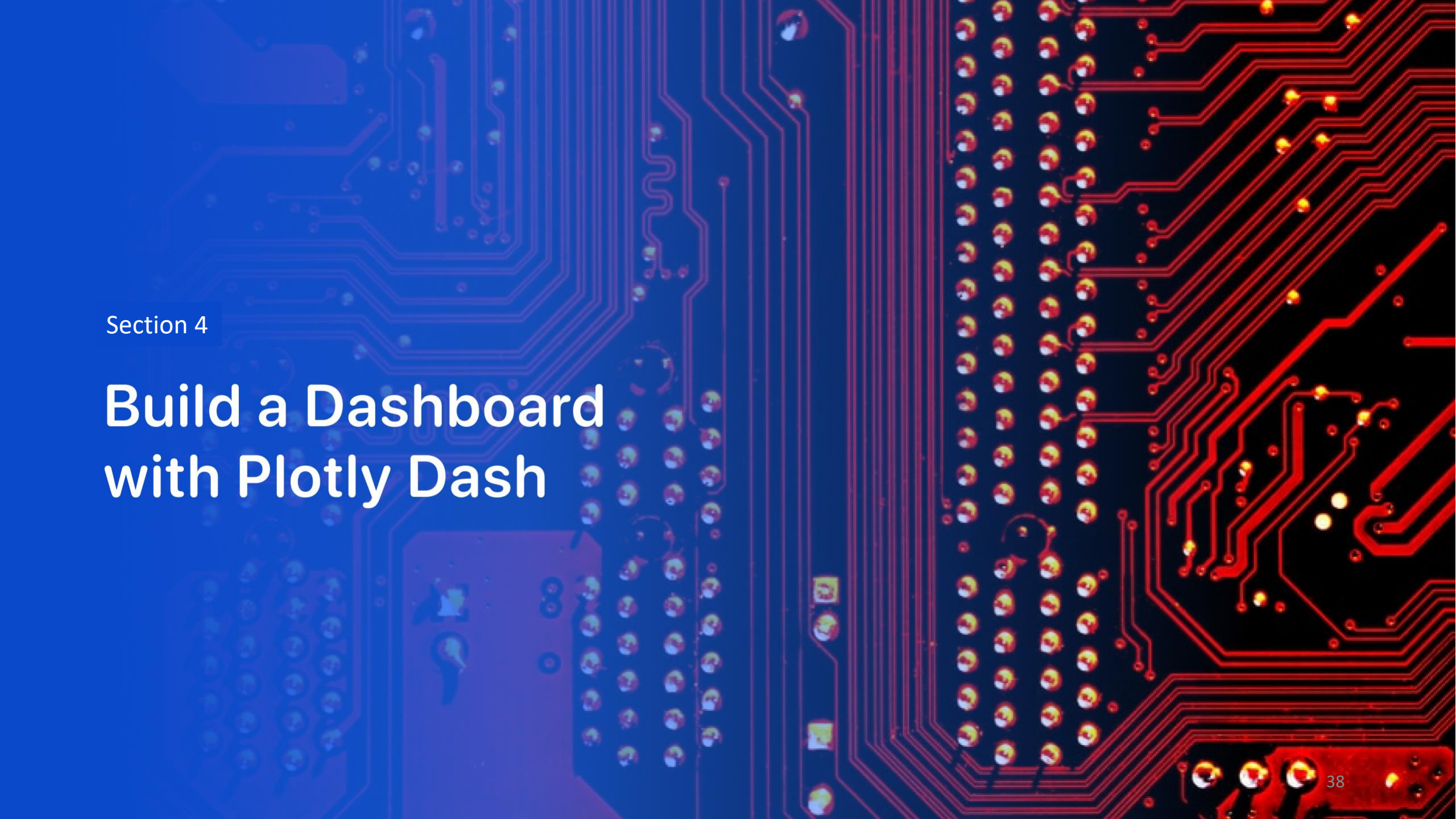


*The views display interactive Outcomes in an easy to absorb format, the details accessed with a point and click. These can be counted in the zoomed in view but by zooming out the number in the group will just be displayed. **It is instantly clear that KSC has a higher success rate than CCAFS for this dataset***

Folium Map: KSC LC-39A View

- **Kennedy SC Launch Complex 39A** stands close to the broad expanse of coast and has water even closer in the bay, with easy emergency access roads. Launches head out over water, staying clear of population dense regions
- Larger roads and railway links are three times further away (Titusville), but provide easy access for tourism. The need here is to balance tourist access with access for emergency services and maintenance vehicles
- Orlando is significantly further away at 78km - well clear of the fireworks with KSC and Cape Canaveral SFC both maintaining a safe distance
- *Note that there are closer two lane roads, one directly through the edge of the complex (Kennedy Parkway North) and another (FL405) crossing over the water towards Titusville. While these are not major (red) roads, they do provide ample access, also to the small regional "Space Coast" airport - **Not Shown in this View***
- *Titusville, and Port Saint John are also population zones at around 20km - a little close for comfort, however not as dense as Orlando, and behind another water barrier towards KSC - **Not Shown in this View***





Section 4

Build a Dashboard with Plotly Dash

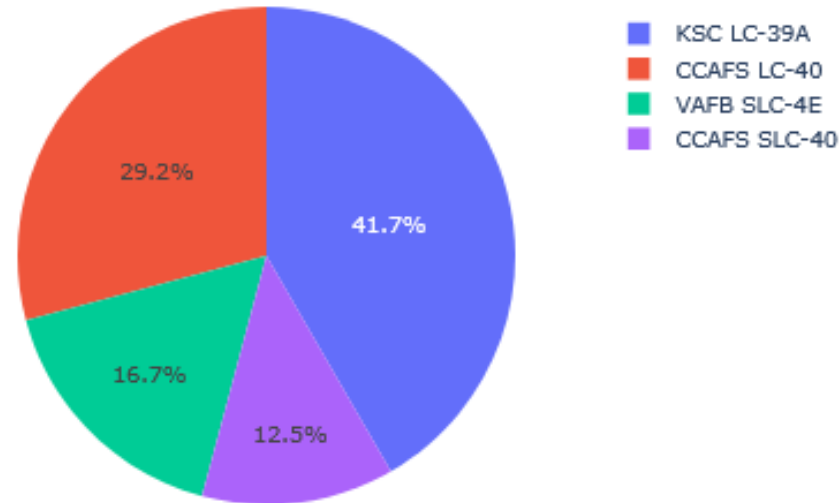
SpaceX Launch Records Dashboard

Total Successful Launches by Site

SpaceX Launch Records Dashboard

All Sites

Total Successful Launches by Site

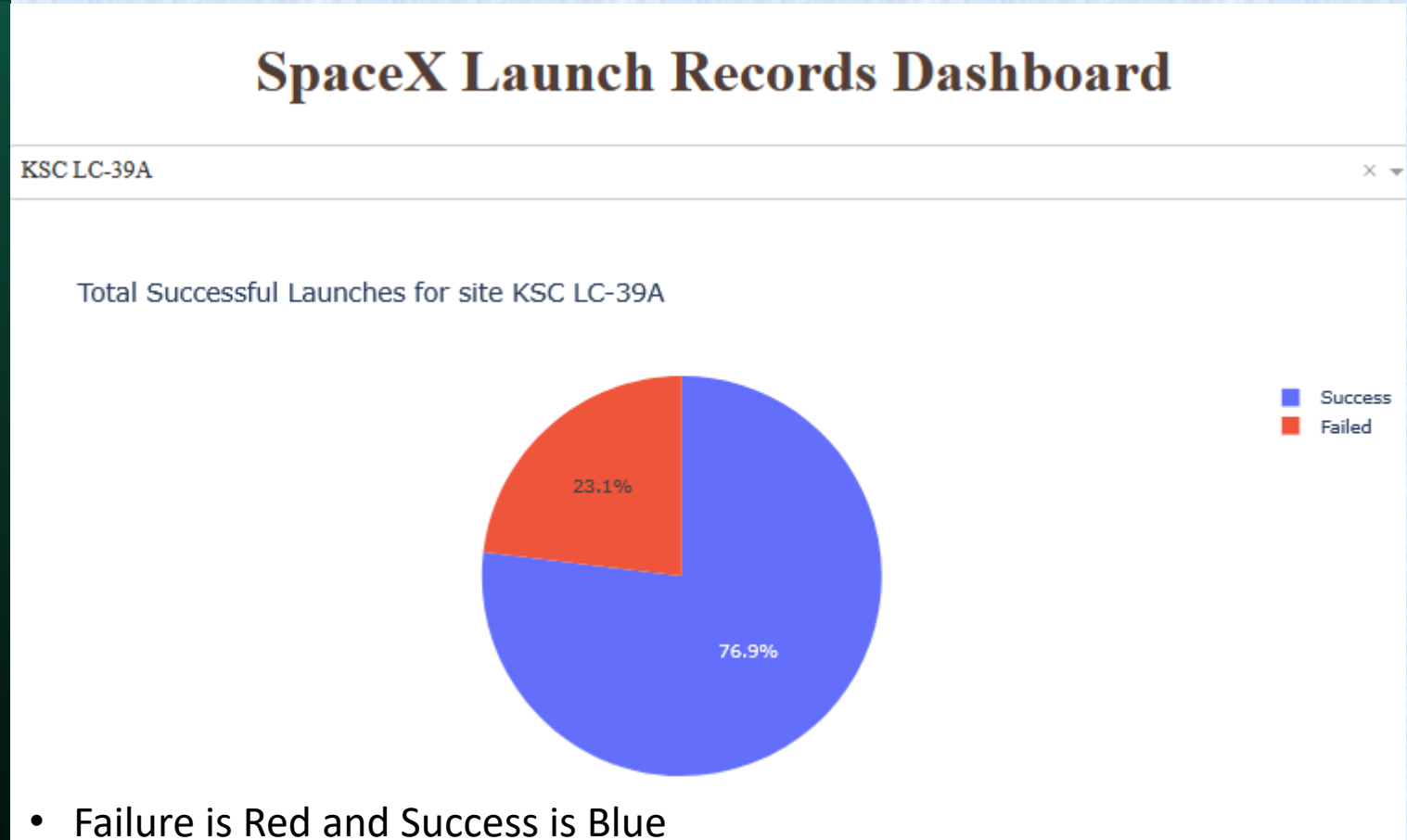


We see 70.9% of all successes launch from just TWO sites: KSC LC-39A and CCAFS LC-40

- The four launch areas are color coded, with Kennedy SC with the top success share, in sky blue (**41.7%**)
- Then Cape Canaveral LC-40 in second place, in orange (**29.2%**)
- Then Vandenberg AFB SLC-4E third, in green (**16.7%**)
- Cape Canaveral SLC-40 is in fourth place, in lilac (**12.5%**)
- The %-ages are of **ALL** the *successful launches* divided up across the 4 areas, not the %-age successes *per site* (e.g. 4/10 successes at VAFB is not 40% of **all** successes)

SpaceX Launch Records Dashboard

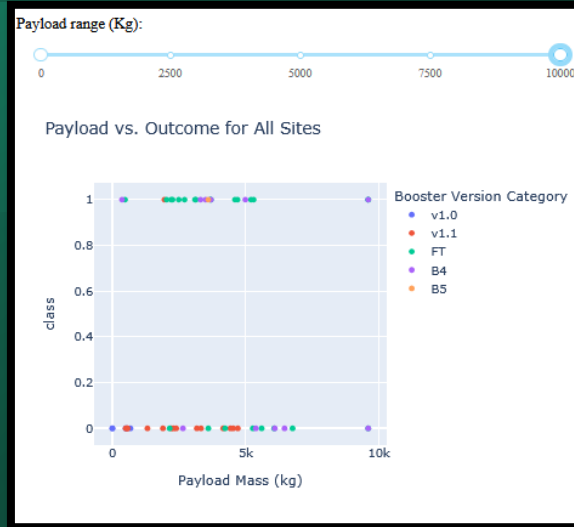
Total Successful Launches for Site KSC LC-39A



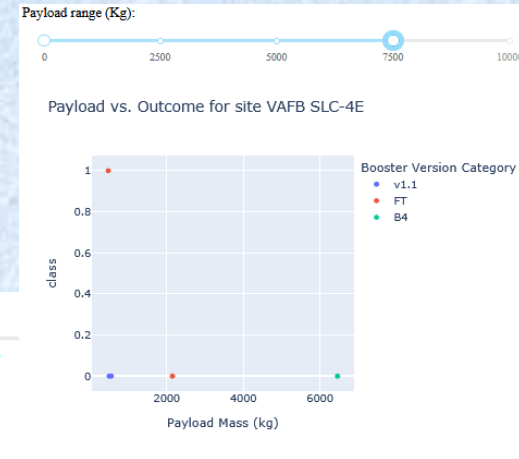
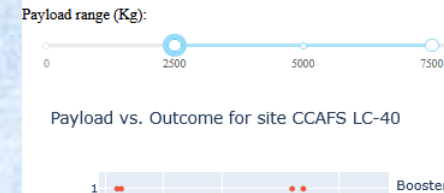
- The %-age is specific to the launch site success rate, and says nothing about the total of all launches across all sites. The dashboard does not show the number of launches or the number of successes, only the %-age
- 76.9% is the highest success rate of any launch site
- Any model predicting successful outcomes by ALWAYS predicting a positive outcome would also score well here! ***It is hard to beat SpaceX at KSC LC-39A***

SpaceX Launch Records Dashboard

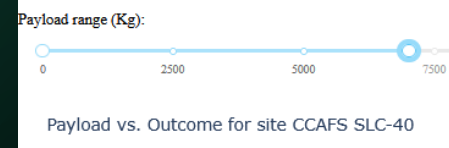
Payload vs. Launch Outcome



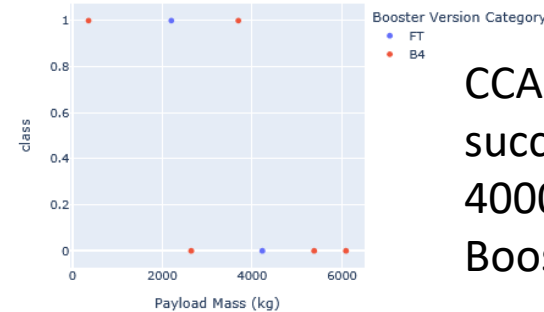
For all Sites/PLoads data, FT boosters do best, B4 are at 50/50, and v1.1 do poorly



VAFB < 7500 PLoad only 1 success (B4) - at lightest PLoad. Then 3 fails (one a B4)



CCAFS LC-40 only FT successes (4) at 2500-7500 PLoad. Just FT and v1.1 Booster here



CCAFS SLC-40 has 3:1 success < 2500. Fails 4000-7000. Only B4/FT Boosters

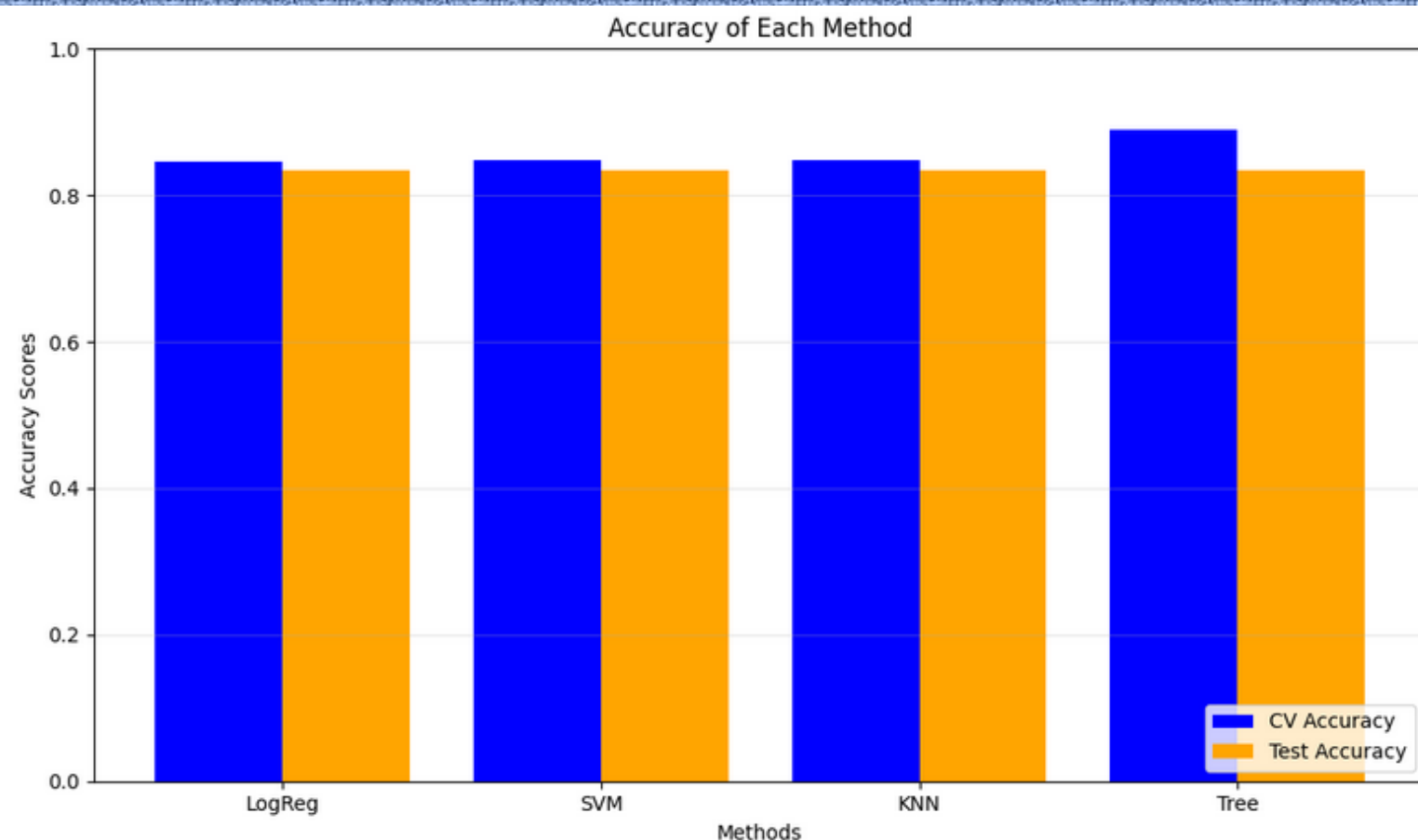
KSC LC-39A: 3 fails of FT Booster > 5500 < 7500; Also FT has 4 success < 5500. B4 booster 3 success < 5500

- *Summary:*
- *VAFB best results > 7500 PL*
- *LC-40 best results with FT Booster*
- *SLC-40 for B4/FT best < 4000*
- *LC-39A best results < 5500*
- *All Sites Combined Data: FT best Booster, then B4*

Section 5

Predictive Analysis (Classification)

Classification Accuracy



	Cross Validation Accuracy	Test Data Accuracy
Tree	0.88929	0.83333
SVM	0.84821	0.83333
KNN	0.84821	0.83333
LogReg	0.84643	0.83333

*Here the Test Data Accuracy comes via final scoring on **unseen** test data, predicting, e.g.:*

```
yhat = tree_cv.predict(X_test)
plot_confusion_matrix(Y_test, yhat)
```

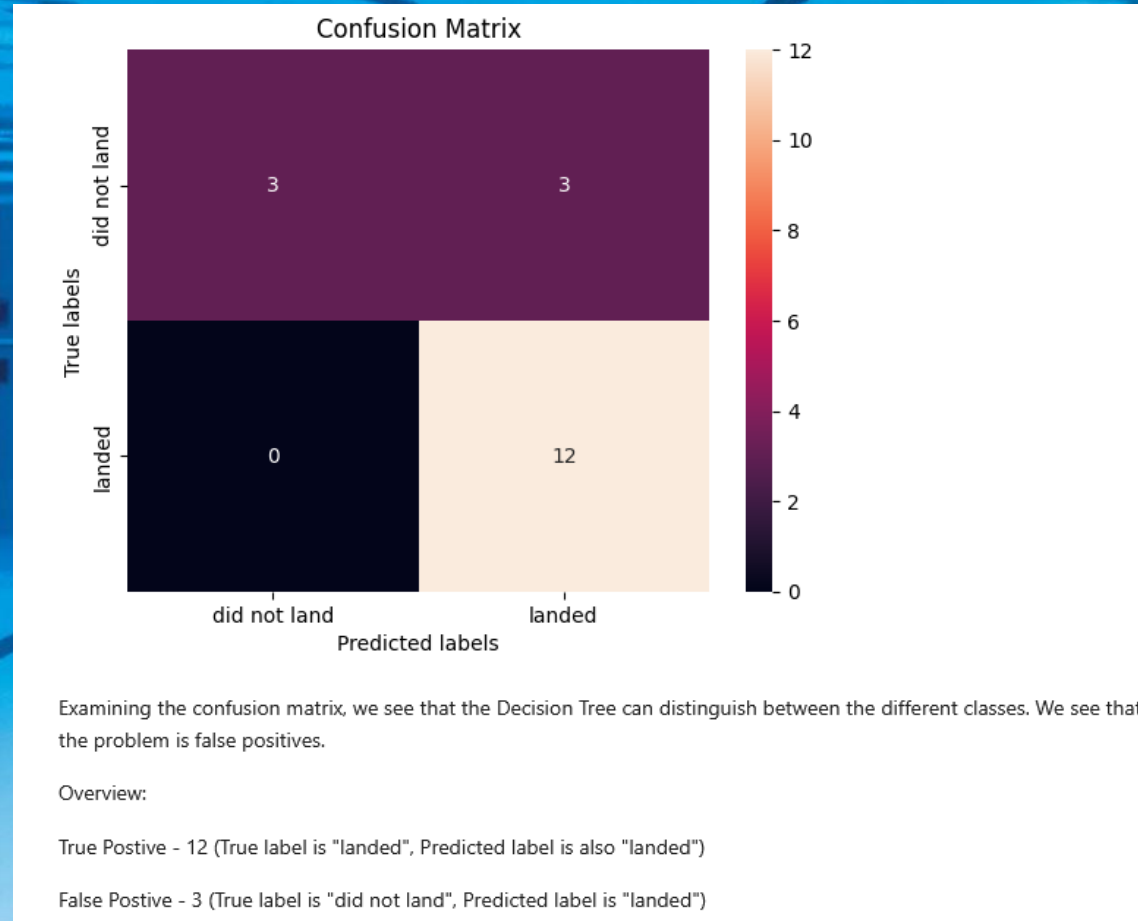
Cross Validation (CV) Accuracy is based on the GridSearchCV output, such as from:

```
print("accuracy :", tree_cv.best_score_)
```


Confusion Matrix

- The confusion matrix is particularly useful for classification problems, and imbalanced datasets, helping evaluate model performance beyond just accuracy
- It also provides metrics like precision, recall, and F1-score, crucial for error analysis and model comparison

- *For probabilistic classifiers, the confusion matrix can be used to evaluate different thresholds for classification, helping to optimize the trade-off between precision and recall*



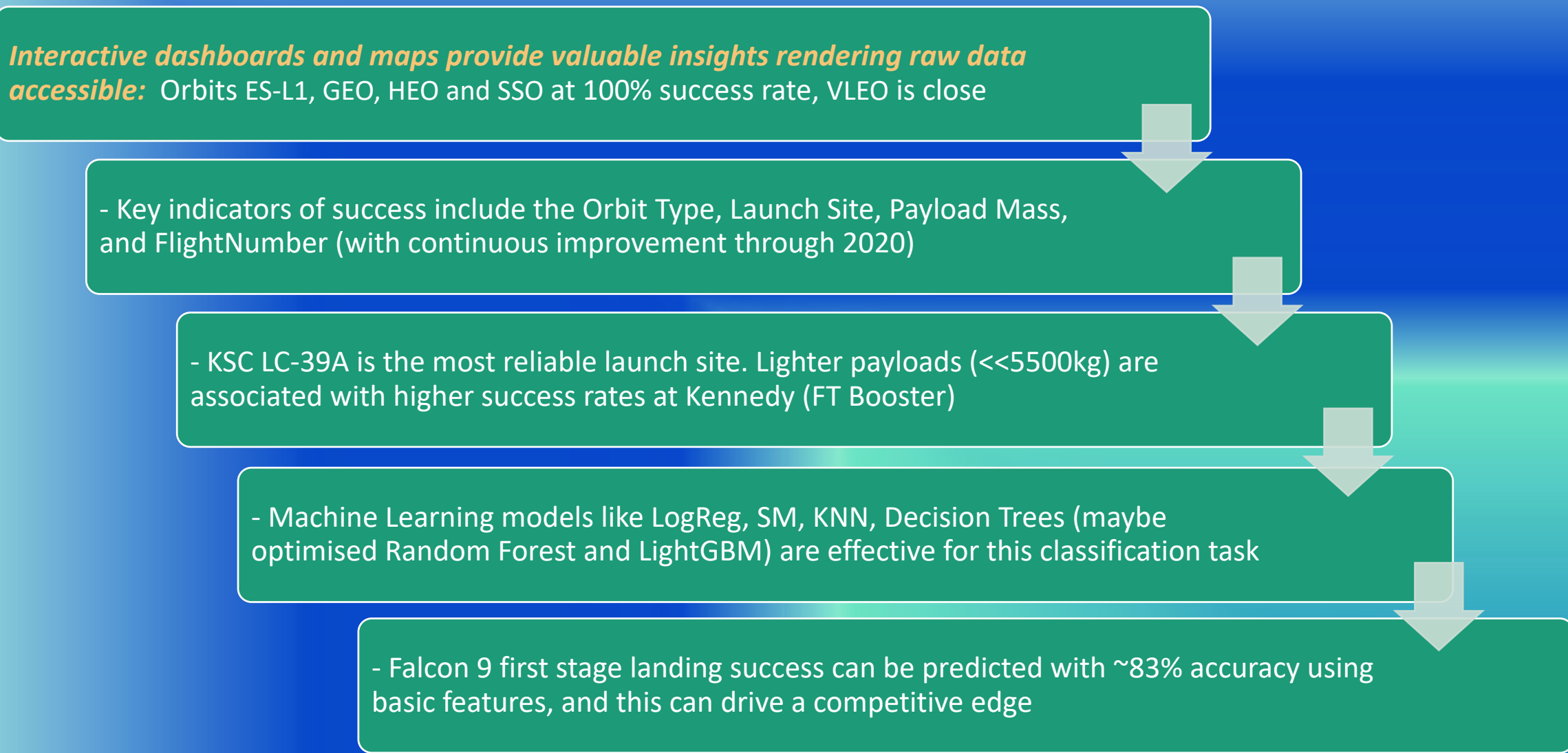
For all four models in initial evaluation the results were identical for the confusion matrix: *The Dataset is Small*

(Note: The C.M. operates on the unseen Test Data)

```
yhat=tree_cv.predict(X_test)  
plotconfusion_matrix(Y_test,yhat)
```

Conclusions

Interactive dashboards and maps provide valuable insights rendering raw data accessible: Orbits ES-L1, GEO, HEO and SSO at 100% success rate, VLEO is close



- Key indicators of success include the Orbit Type, Launch Site, Payload Mass, and FlightNumber (with continuous improvement through 2020)

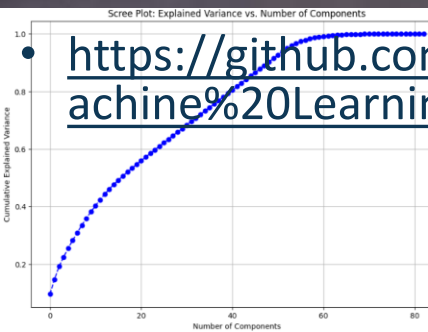
- KSC LC-39A is the most reliable launch site. Lighter payloads (<<5500kg) are associated with higher success rates at Kennedy (FT Booster)

- Machine Learning models like LogReg, SM, KNN, Decision Trees (maybe optimised Random Forest and LightGBM) are effective for this classification task

- Falcon 9 first stage landing success can be predicted with ~83% accuracy using basic features, and this can drive a competitive edge

References

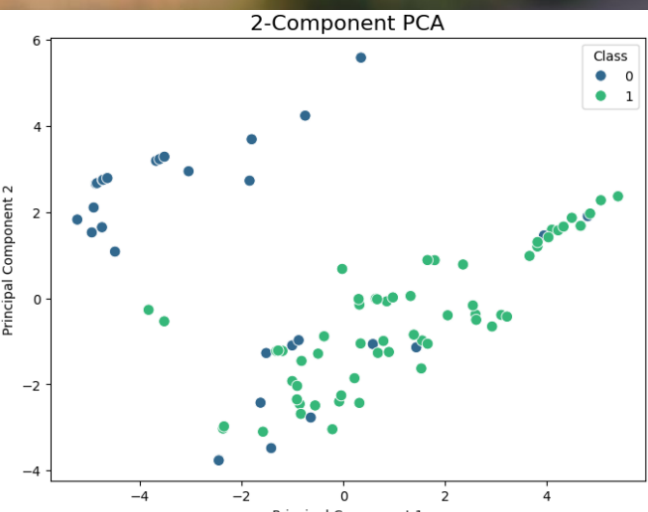
- <https://github.com/CallowEnder/TestDataScience/blob/main/jupyter-labs-spacex-data-collection-api.ipynb>
- <https://github.com/CallowEnder/TestDataScience/blob/main/jupyter-labs-webscraping.ipynb>
- <https://github.com/CallowEnder/TestDataScience/blob/main/labs-jupyter-spacex-Data%20wrangling.ipynb>
- https://github.com/CallowEnder/TestDataScience/blob/main/jupyter-labs-eda-sql-coursera_sqlite.ipynb
- <https://github.com/CallowEnder/TestDataScience/blob/main/edadataviz.ipynb>
- https://github.com/CallowEnder/TestDataScience/blob/main/lab_jupyter_launch_site_location.ipynb
- https://github.com/CallowEnder/TestDataScience/blob/main/spacex_dash_app.py
- https://github.com/CallowEnder/TestDataScience/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5.ipynb



Resources

Project Resources:

- GitHub Repository: <https://github.com/CallowEnder/TestDataScience/>
- Project Report: See 'General Report.pdf' in the repository:
<https://github.com/CallowEnder/TestDataScience/blob/main/General%20Report.pdf>
- Interactive Dash App: https://github.com/CallowEnder/TestDataScience/blob/main/spacex_dash_app.py



Research

Findings link at GitHub -
some work on Random Forests &
LightGBM & Optimization



Appendix

The enhanced Decision Tree models researched, do represent the best possible theoretical choices for such imbalanced and sparse datasets. We saw how little difference there was from the results. However we need to ask how well we can explain these models results in Data Storytelling. Stakeholders know what they know. Selecting the simplest model makes sense then.

I could fill the Appendix with explanations and contrasts but but add no value. More data is our real need if we are to do better.



Thank you!

